

STRATIV

AI Receipt Scanner

System Documentation

Audience Operations, IT and technical handover

System Android application, internal distribution, Expo SDK 57

App version 1.1.0 · Update channel `main`

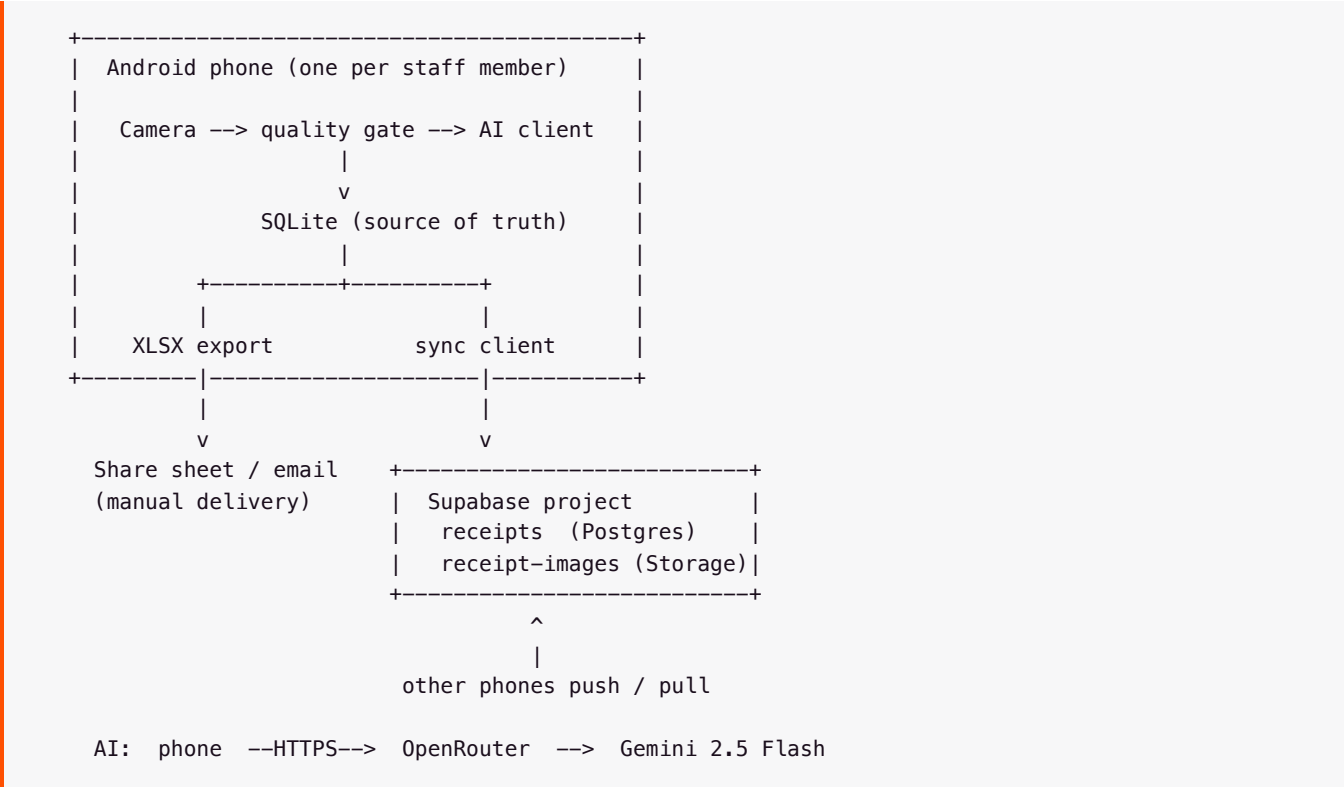
Document date 20 August 2026

Source docs/system-documentation.html — edit the HTML, regenerate the PDF

1. System overview

AI Receipt Scanner digitises paper expense receipts for a Strativ office. It runs entirely on staff phones, calls one external AI service to read each receipt, optionally mirrors the results to a shared database so the team sees one another's scans, and produces a monthly Excel **Bill Approval Sheet** for sign-off.

It is not a server-side system. There is no backend to operate, no cron, no queue, and no admin panel. The operational surface is: the app on each phone, one AI vendor account, and one Supabase project.



Design principles

Principle	Consequence
Local-first	SQLite is authoritative. The app is fully usable with no connectivity except during the AI call itself.
Failure must not cost data	Sync is fire-and-forget and retried. A sync failure is never presented as a failed scan.
Additive sync only	The pull adds and refreshes; it never deletes local rows. There is no delete permission on the remote at all.
Monthly deliverable	The output is one signed document per month, not a live feed.

2. Components

Component	Responsibility	Failure behaviour
Quality gate utils/image-quality.ts	Rejects blurry, dark or non-receipt photos on-device before any API call. Runs on a 160 px thumbnail.	If it cannot analyse the image it passes the photo through rather than blocking the user. Staff can always override.
AI client services/ai-vision.ts	Sends the compressed image to OpenRouter, receives structured JSON, normalises it.	Never throws. Returns an error object the capture screen renders. Retries with backoff; aborts immediately on 401/402.
Local store services/storage.ts	SQLite: receipts, config, export columns. Statistics and month queries.	Retries a stale connection (a known Android <code>execAsync</code> null-pointer condition) by reopening the database.
Sync client services/remote-db.ts	Pushes rows and photos to Supabase, pulls the office's rows back.	Silent and non-fatal. Unsent receipts stay marked pending and go on the next app launch.
Sheet generator services/xlsx-export.ts	Builds the Bill Approval Sheet from a month's receipts and the column config.	Purely local and deterministic. No network dependency.
OTA client hooks/use-ota-updates.ts	Checks for a JavaScript update 3 s after launch; manual check in Settings.	Silent on failure; the installed bundle keeps running. Disabled in dev builds.

3. Data model

On device — SQLite

Database `receipts.db` , three tables.

Column	Type	Notes
<code>id</code>	TEXT PK	<code>receipt_<timestamp>_<random></code> . Generated on device; also the remote primary key.
<code>merchant_name</code>	TEXT	Nullable. May hold Bangla text.
<code>receipt_date</code>	TEXT	ISO <code>YYYY-MM-DD</code> , or null when unreadable.
<code>receipt_number</code>	TEXT	Nullable.
<code>invoice_type</code>	TEXT	retail, restaurant, utility, service, unknown.
<code>items</code>	TEXT	JSON array of line items.
<code>subtotal</code> , <code>tax</code>	REAL	Nullable.
<code>total</code>	REAL NOT NULL	Required — a receipt with no total is not accepted.
<code>currency</code>	TEXT	Defaults to BDT.
<code>payment_method</code>	TEXT	Nullable.
<code>confidence_score</code>	REAL	0–1, recalibrated against field completeness and arithmetic consistency.
<code>image_uri</code>	TEXT NOT NULL	<code>file://</code> = scanned on this device. An <code>https://</code> bucket URL = pulled from the database. Empty = pulled, photo unavailable.
<code>raw_text</code> , <code>error_message</code>	TEXT	Diagnostics.
<code>created_at</code>	TEXT NOT NULL	ISO timestamp.
<code>synced_at</code>	TEXT	Null means not yet in the shared database. This single column drives all offline catch-up.

`config` stores key/value settings (office name, optional endpoint). `export_columns` stores which sheet columns are enabled and in what order. Indexes exist on `created_at` , `invoice_type` , `merchant_name` and `substr(receipt_date,1,7)` for month queries.

WHY RECEIPT_DATE CAN BE NULL

Handwritten receipts sometimes have no legible date. Month queries use `COALESCE` so those receipts appear in the month they were *scanned* rather than falling out of every month's sheet — which is what happened before, and meant expenses silently disappeared.

Shared — Supabase

Defined and re-runnable in `supabase/schema.sql`.

Object	Purpose
<code>receipts</code> table	Mirrors the device columns, plus <code>office_name</code> (one project can serve several offices) and <code>image_path</code> (object path of the photo). Upserted on <code>id</code> .
<code>receipt-images</code> bucket	Private. One object per receipt, <code><receipt id>.jpg</code> , 5 MB per-file limit.

Row-level security is enabled and the app holds the anon role only:

Operation	<code>receipts</code> table	<code>receipt-images</code> bucket
insert	Allowed	Allowed
update	Allowed (needed for upsert)	Allowed (needed for replace-on-reupload)
select	Allowed — this is what makes the team view work	Allowed
delete	No policy	No policy

The missing delete policy is the single most important line in the schema: a leaked key cannot erase the office's expense records. Deletion requires the Supabase dashboard or the service key, neither of which is on any phone.

4. Synchronisation

Push

1. A receipt is saved locally with `synced_at` `null`.
2. The photo is resized (1600 px, quality 0.7) and uploaded to the bucket; the returned object path goes on the row.
3. The row is upserted on `id` with `resolution=merge-duplicates`, so re-sending never duplicates.
4. Only ids the endpoint actually accepted get `synced_at` stamped. Everything else is retried on the next launch.

Editing a receipt clears `synced_at`, so corrections propagate. A photo that fails to upload does not hold back its row — the record syncs with no photo, and Settings → Data → Re-upload all receipts sends it later.

Pull

On app start the device fetches the office's receipts (newest first, 2000-row limit) and merges them. The **ownership rule** decides each row:

Local state	Action
Row absent	Insert it. Another phone's scan appears here.
Row present, <code>image_uri</code> starts <code>file://</code>	Skip . This device scanned it and owns it.
Row present, came from the database	Refresh it, so a correction made on the owning phone arrives.

NEVER DELETE LOCAL ROWS TO MATCH THE REMOTE

An empty read is indistinguishable from a blocked, rate-limited or failed one. "The table looks empty, so clear the phone" would destroy the office's records, including scans that had not uploaded yet. If the remote table is emptied by accident, recover the other direction: Settings → Data → Re-upload all receipts clears `synced_at` and pushes everything back.

Consequences to be aware of

- Sync is **eventual**. Two phones agree after both have opened the app with a connection, not immediately.
- There is **no conflict resolution**. If the same receipt is edited on two phones, the last push wins. In practice one person owns a receipt, so this has not occurred.
- Photos load over the network on the phones that did not take them, so the History list on those devices needs a connection to show thumbnails.
- Running **without** Supabase configured is fully supported. The app is then single-device and every sheet covers one phone's scans.

5. External dependencies

Service	Used for	If unavailable	Cost model
OpenRouter → Gemini 2.5 Flash	Reading each receipt	Scanning stops working. Everything else — history, editing, export — continues. This is the only hard external dependency.	Per request. Use a spend-capped key.
Supabase (Postgres + Storage)	Shared team view, photo backup	Degrades to single-device. Scans queue locally and upload later. No data loss.	Free tier is sufficient at current volume; see \$9.
EAS (Expo Application Services)	Builds and OTA updates	No effect on installed apps. Only blocks shipping changes.	Expo plan.

6. Security model

What is exposed

Every `EXP0_PUBLIC_*` value is compiled into the JavaScript bundle. Anyone who obtains the APK can extract the OpenRouter key and the Supabase anon key. This is inherent to a client-only app with no login, and is accepted for internal distribution. The mitigations are structural rather than secret-based:

Risk	Mitigation
Extracted OpenRouter key used to spend money	Spend cap on the key. Rotate via MAINTENANCE.md §3 — a rotation is an OTA update, not a rebuild.
Extracted anon key used to read expense data	Accepted. The data is internal merchant names, dates and amounts. Add Supabase Auth if that changes.
Extracted anon key used to destroy data	Not possible. No delete policy exists on the table or the bucket.
Receipt photos read by an outsider	The bucket is private; reads require the key. Public URLs were rejected because receipt ids embed a timestamp and would be partly guessable.
Lost or stolen phone	Receipts already synced are safe in the shared database. Unsynced scans on that device are lost. There is no remote wipe.

Data handling

- Receipt images are sent to OpenRouter for extraction. Treat that as processing by a third party when assessing what may be photographed.
- Photos are stored twice: on the capturing device and in the private bucket.
- The app collects no personal data about its users — no accounts, no analytics, no tracking.
- Android permissions requested: camera, and read/write external storage for importing and sharing files.

7. Environments and configuration

Variable	Where it is set	Effect if missing
EXP0_PUBLIC_OPENROUTER_API_KEY	.env locally; EAS environment variable for builds	Every scan fails.
EXP0_PUBLIC_SUPABASE_URL	Same	App runs local-only; no team view.
EXP0_PUBLIC_SUPABASE_ANON_KEY	Same	If the URL is set but the key is not, every sync is rejected. The app logs a warning at start.

The database endpoint is **build configuration, not a user setting**. There is deliberately no field, status indicator or sync button in the app: a phone is pointed at the office project by installing the build, and nothing else. Staff have no reason to see a database URL, and every extra field is another way to break sync.

Profiles and channels

EAS profile	Channel	Environment	Use
development	development	development	Dev client for local work.
preview	main	preview	The build staff install. Internal distribution.
production	production	production	Store-oriented profile. Unused — this app is not published.

8. Build, release and update

Which mechanism applies

Change	Mechanism
JavaScript, TypeScript, styling, prompt, sheet layout	OTA update. Reaches phones on next launch.
API key rotation	OTA update (the key is read from the bundle).
New native module or config plugin, app.json native change, SDK upgrade	New build. Each phone reinstalls the APK.
Database column	SQL first, then OTA. See the warning below.

```
# Ship a JavaScript change to the phones staff use
eas update --channel main --environment preview -p android -m "what changed"

# Build a new APK when native code changed
eas build --platform android --profile preview
```

ORDER MATTERS FOR SCHEMA CHANGES

Apply `supabase/schema.sql` **before** publishing an update that references a new column. PostgREST rejects a payload naming an unknown column, so the wrong order does not degrade sync — it stops it completely, for every device, until the SQL is run.

An OTA update only reaches builds with a matching **runtime version**, and the policy is `fingerprint` — the runtime version is derived from the app's native dependencies rather than its version number. An update is therefore only ever offered to builds with the same native shell, and a rebuild is required whenever those dependencies change. Exact procedures: MAINTENANCE.md §1 and §2.

WHY THE POLICY IS NOT APPVERSION

It was, until the SDK 57 upgrade. Under `appVersion` the runtime version is the version string, so an SDK 54 build and an SDK 57 build both claimed `1.0.0` and sat on channel `main` together — one OTA would have been served to both populations, and a bundle built against React Native 0.86 crashes on launch inside an 0.81 shell. Nothing warned about it; the versions simply matched. `fingerprint` makes that combination impossible rather than merely discouraged.

9. Operations

The monthly cycle

1. Staff scan receipts as they arrive, throughout the month. No network needed beyond the scan itself.
2. Corrections are made on the phone that scanned the receipt, via the detail screen.
3. At month end, one person opens **Export**, picks the month, enters the amount received from the account and the four signature names, and generates the sheet.
4. The sheet is shared from the phone (email, chat, or file share) and printed for signature.

Before generating, confirm every phone has opened the app with a connection — otherwise a colleague's unsynced scans will be missing from the sheet.

Runbook

Symptom	Cause and action
"OpenRouter API key not found"	Key missing from the build, or the dev server started before the variable existed. Restart with <code>pnpm start --clear</code> ; for a build, check the EAS environment variable and publish an update.
Scans fail with 402	OpenRouter credits exhausted. Top up or rotate the key (MAINTENANCE.md §3).
Scans fail with 401	Invalid or revoked key. The app stops retrying immediately by design.
A colleague's receipts are missing	Their phone has not synced. Have them open the app with a connection; the backlog uploads automatically. If it persists, check the Supabase URL and anon key are both present in the build.
Photos show as blank thumbnails	Expected for receipts scanned before photo sync existed, and for any upload that failed. Settings → Data → Re-upload all receipts on the owning phone backfills them.
Nothing syncs at all, from any device	Almost always a schema mismatch — an app update shipped ahead of its SQL. Apply <code>supabase/schema.sql</code> .
Remote table was emptied by accident	Do not clear any phone. Run Settings → Data → Re-upload all receipts on each device.
Bangla text is unreadable in an export	Confirm the file is <code>.xlsx</code> . CSV cannot carry the encoding reliably and was removed for this reason.
Blurry photos being accepted	The quality gate is threshold-based and can be recalibrated in <code>utils/image-quality.ts</code> . Constants are at the top of the file.
App will not open after an update	Reinstall the APK. The previous OTA bundle is replaced, not kept, so rollback means publishing a corrected update.

Capacity and limits

Limit	Value	When it matters
Photo storage	~250 KB per receipt after resize; Supabase free tier is 1 GB	Roughly 4,000 receipts. Full-resolution frames would have exhausted it in a few hundred.
Per-file bucket limit	5 MB	Never reached after resizing.
Pull size	2,000 newest rows	A team accumulating more will stop seeing the oldest. Raise the limit in <code>fetchRemoteReceipts</code> .
Push batch	500 pending rows per pass	Only relevant after a long offline period; the next launch continues.
Re-upload bound	20 passes, so 10,000 receipts	A hard stop, not a target. Documented in <code>services/config.ts</code> .
Image accepted for scanning	≤ 20 MB file, ≤ 33 megapixels, compressed to ≤ 4 MB for the API	Modern phone cameras are within these.

10. Known limitations

- **Eventual consistency, no conflict resolution.** Simultaneous edits on two phones resolve last-write-wins.
- **No authentication.** Possession of the app is possession of the office's expense data.
- **No remote wipe** for a lost device, and unsynced scans on it are unrecoverable.
- **No backup automation.** Supabase's own backups are the only copy beyond the phones. Nothing exports the table on a schedule.
- **Delete-all is available in Settings** and is protected only by a two-tap confirmation. Consider hiding it before wider rollout.
- **iOS is configured but untested.** Bundle identifier and permissions exist; no iOS build has been distributed.
- **Accuracy is not guaranteed.** Confidence is reported per receipt and low-confidence scans are flagged, but a human check before approval remains part of the process. That is what the signature blocks are for.

11. Related documents

Document	Contents
docs/developer-guide.html	Codebase structure, module reference, conventions, testing, known traps.
MAINTENANCE.md	Living runbook with exact commands: builds, OTA, key rotation, schema changes, release notes. The authority for procedures.
supabase/schema.sql	The shared database definition. Re-runnable.
RELEASE_NOTES.docx	What changed in each release, newest first.